

AI INTERN ASSIGNMENT

Legal Document Generation & Evaluation Agent

1. Overview

Legal professionals work with highly structured documents such as Affidavits, Written Arguments, Plaints, Applications, Indexes and Memoranda. These documents follow recurring structural and formatting patterns. Generating correct text is not enough: the information must appear in the correct place, and the finished document must stay consistent with the required format.

Your task is to build a small AI-powered Legal Document Generation Agent. It takes a sample legal document as a reference, accepts case-specific information, generates a new document following the reference structure, and then evaluates the generated document for accuracy, completeness and consistency.

2. Problem Statement

For this assignment you will work with an Affidavit in Reply. You are given a reference document and a separate set of case information. Your system should use the reference to understand the expected structure and produce a new Affidavit in Reply populated with the supplied information.

The purpose is not to test legal knowledge. All legal facts, entities and substantive content are supplied. The focus is on building a reliable AI workflow for extraction, mapping, generation and evaluation.

3. Materials Provided

File	What it is
01_Affidavit format explained	A plain-language description of the Affidavit in Reply structure, the fixed phrases used, and an entity list. Use it as a glossary and a checklist.
02 Affidavit in reply sample	A complete Affidavit in Reply from an unrelated matter. This is your format reference.
03 Case_Information	The facts, parties and reply points for the document you must generate.

4. Core Task

Build a modular AI workflow with the following stages:

Stage	What it does
Template / document understanding	Identify the major sections and recurring structure of the reference document.
Entity extraction	Extract court, jurisdiction, case number, year, petitioner, respondent, respondent number, deponent, capacity, address, dates, exhibits and advocate details into a structured form.
Content mapping	Map the supplied case information to the correct sections of the reference format.
Document generation	Generate the new Affidavit in Reply using the supplied information while preserving the reference structure.
Validation / evaluation	Check the generated document against the ground truth defined in Section 4 and identify errors.
Evaluation report	Produce a concise report containing scores and detected issues.

5. Scope — Keep It Small

You are not expected to build a legal drafting platform. For this assignment:

- Support one document type only: Affidavit in Reply.
- Use the supplied reference document rather than designing a format from scratch.
- Use the supplied case facts and reply points. Do not do independent legal research.
- A para-wise reply (answering the petition paragraph by paragraph) is out of scope.
- You may use any LLM, Python library, framework or document-generation approach.
- The goal is a working proof of concept, not a production system.

6. Expected Output

The generated document should broadly preserve the reference format, including:

- Forum and jurisdiction heading
- Case / cause number
- Cause title and party labels
- Affidavit title naming the answering respondent
- Deponent clause
- Numbered reply paragraphs
- Exhibit reference
- Prayer

- Jurat / attestation
- Verification
- Advocate / drafting block

You are not expected to reproduce every visual characteristic of a professionally typeset court filing. The document should, however, show deliberate preservation of structure, headings, numbering, bold conventions and paragraph organisation.

7. Evaluation Requirement

The generated document must not simply be displayed as the final answer. Your system must also evaluate its own output. Build a basic evaluation mechanism that compares the generated document against the ground truth defined in Section 4.

Evaluation area	What to check	Example
Entity accuracy	Are extracted and inserted entities correct?	Respondent No. 2 written as Respondent No. 3
Completeness	Is required information missing?	Deponent designation omitted
Structure	Are required sections present and in the expected order?	Verification block missing
Consistency	Are related fields consistent throughout?	Respondent number differs between title and jurat
Template fidelity	Does the output follow the reference structure?	Prayer numbered instead of lettered
Hallucination	Has information been introduced that was not supplied?	Invented party, date or statutory provision

8. Evaluation Report

Your workflow should produce a short evaluation report. The exact scoring method is up to you — any defensible scheme is acceptable, provided you explain it. It should contain:

- An overall score.
- Individual scores for the major evaluation dimensions.
- A list of errors and issues detected.
- The source or reference for each detected issue, where practical.
- A short explanation of how the score was calculated.

Illustrative only — these numbers are not a target:

Overall Score: 91/100

Entity Accuracy: 95/100
Completeness: 90/100
Structure: 100/100
Consistency: 88/100
Template Fidelity: 84/100
Hallucination Check: 100/100

Issues Detected:

1. Respondent number is inconsistent in paragraph 5.
2. Exhibit A is referenced but not mapped correctly.
3. Verification paragraph range does not match the body.

9. Suggested Workflow

```
Reference Document
|
Template / Structure Analysis
|
Case Information  ->  Entity Extraction
|
Content Mapping
|
Document Generation
|
Generated Affidavit in Reply
|
Entity / Structure Extraction
|
Ground Truth Comparison
|
Evaluation Score + Error Report
```

10. Deliverables

Submit all five of the following.

1. GitHub repository

- A public repository, or a private one with access granted to the reviewer email given in the covering mail.
- All source code, committed with a readable history. A single squashed commit at the deadline is acceptable but not preferred.
- An /outputs folder containing the generated Affidavit in Reply (preferably .docx) and the evaluation report (.md or .json).
- Do not commit API keys. Use a .env file, ignored by git, and include a .env.example listing the variable names.

2. Working link

- A link where the reviewer can run the system end to end without installing anything — for example Streamlit Community Cloud, Hugging Face Spaces, Render, Replit or a shared Google Colab notebook set to "anyone with the link".
- The link must actually run. If it depends on an API key you cannot expose, provide either a key-entry field in the interface or a pre-run demo mode using cached output, and say so in the README.
- Free hosting that sleeps when idle is fine. Note the wake-up delay in the README.

3. README file

A README.md at the repository root containing:

- What the project does, in three or four sentences.
- Setup instructions: Python version, dependency install command, environment variables required.
- How to run it — the exact commands, in order, from a clean clone.
- The working link and the video link.
- An architecture or workflow diagram. A Mermaid block or an embedded image is fine.
- Your design decisions: what approach you chose and why, and what you rejected.
- Known limitations and failure cases.
- If you used an AI coding assistant, a line saying where.

4. Video demo

- Under 5 min, Loom, or Google Drive link with link-sharing enabled. Put the link in the README.
- Show the system running end to end: inputs going in, the generated document coming out, and the evaluation report.
- Spend the last minute on one thing your validation layer catches and how it catches it.
- Screen recording with voice-over is enough. Production quality is not assessed.

5. Generated artefacts

- The generated Affidavit in Reply for the supplied case information.
- The evaluation report for that document.

Both committed in the repository, not sent separately.

11. Timeline and Submission

Item	Detail
Time allowed	5 calendar days from the date and time this assignment is sent to you.

Item	Detail
Deadline	11:59 PM IST on the fifth day.
Expected effort	Approximately 8 to 12 hours. Please do not spend more.
How to submit	Reply to the assignment email with: the GitHub repository link, the working link, and the video link.
Extensions	Ask before the deadline, not after. A short extension is usually possible if you say why.

If you run out of time, submit what works. A clean partial solution with an honest README explaining what is unfinished scores better than a broken complete one.

12. Minimum Bar and Bonus

Must have — a submission with all of these is complete

- Reads the reference document and the case information.
- Extracts entities into a structured intermediate form.
- Generates the Affidavit in Reply following the reference structure.
- Runs at least three deterministic checks that do not depend on an LLM. For example: the respondent number is consistent throughout; the verification paragraph range matches the number of body paragraphs; every required section is present.
- Produces an evaluation report with scores and an issue list.
- Repository, working link, README and video.

Nice to have — additional credit

- A validation layer that catches errors before the final document is produced, not only after.
- A structured intermediate JSON representation between extraction and generation.
- Evidence mapping showing where each piece of information came from.
- A reusable template or schema approach rather than hard-coded fields.
- A mechanism that explains why the system failed, not only that it failed.
- Handling of a second document type, demonstrated or designed for.

13. Tools and Permissions

- Any LLM is permitted — hosted or local. Free tiers are entirely acceptable and no candidate is expected to pay for API access.
- AI coding assistants are permitted and expected. Note in the README where you used one. We are more interested in your judgement than your typing.

- Any framework, library or document-generation approach is permitted.
- The materials in this pack are anonymised sample documents and contain no real party data.